

```

import random

def create_board():
    return [[' ' for _ in range(3)] for _ in range(3)]

def print_board(board):
    print("\n 0 1 2")
    for idx, row in enumerate(board):
        print(f"{idx} " + " | ".join(row))
        if idx < 2:
            print(" ----+----")
    print()

def check_win(board, symbol):
    for i in range(3):
        if all(board[i][j] == symbol for j in range(3)) or \
            all(board[j][i] == symbol for j in range(3)):
            return True

    if all(board[i][i] == symbol for i in range(3)) or \
        all(board[i][2 - i] == symbol for i in range(3)):
        return True

    return False

def check_draw(board):
    return all(cell != ' ' for row in board for cell in row)

def get_winning_or_blocking_move(board, symbol):
    lines = [
        [(0,0), (0,1), (0,2)], [(1,0), (1,1), (1,2)], [(2,0), (2,1), (2,2)],
        [(0,0), (1,0), (2,0)], [(0,1), (1,1), (2,1)], [(0,2), (1,2), (2,2)],
        [(0,0), (1,1), (2,2)], [(0,2), (1,1), (2,0)]
    ]

    for line in lines:
        symbols_in_line = [board[r][c] for r, c in line]
        if symbols_in_line.count(symbol) == 2 and symbols_in_line.count(' ') == 1:
            for r, c in line:
                if board[r][c] == ' ':
                    return r, c

    return None

def computer_move(board, computer_symbol, human_symbol):
    win_move = get_winning_or_blocking_move(board, computer_symbol)
    if win_move:
        return win_move

    if all(cell == ' ' for row in board for cell in row):
        return random.choice([(r, c) for r in range(3) for c in range(3)])

    block_move = get_winning_or_blocking_move(board, human_symbol)
    if block_move:
        return block_move

    empty_cells = [(r, c) for r in range(3) for c in range(3) if board[r][c] == ' ']
    return random.choice(empty_cells)

def main():
    board = create_board()
    human_symbol = 'O'
    computer_symbol = 'X'
    current_turn = 'Human'
    game_status = 'Active'

    print("=== Tic-Tac-Toe ===")

    while game_status == 'Active':
        print_board(board)

        if current_turn == 'Human':
            while True:
                try:
                    user_input = input("Enter position as 'row col' (e.g., 0 2): ").strip()

```

```

        row, col = map(int, user_input.split())

        if 0 <= row <= 2 and 0 <= col <= 2:
            if board[row][col] == ' ':
                break
            else:
                print("Position is not empty! Try again.")
        else:
            print("Invalid range! Numbers must be between 0 and 2.")
    except ValueError:
        print("Invalid format! Please enter two numbers separated by a space.")

    board[row][col] = human_symbol
    active_symbol = human_symbol

    else:
        print("Computer is making a move...")
        row, col = computer_move(board, computer_symbol, human_symbol)
        board[row][col] = computer_symbol
        active_symbol = computer_symbol

    if check_win(board, active_symbol):
        print_board(board)
        print(f"Game Over! Winner: {current_turn}")
        game_status = 'Win'
        break

    if check_draw(board):
        print_board(board)
        print("Game Over! It's a Draw.")
        game_status = 'Draw'
        break

    current_turn = 'Computer' if current_turn == 'Human' else 'Human'

if __name__ == "__main__":
    main()

```

=== Tic-Tac-Toe ===

```

 0 1 2
0  |  |
---+---
1  |  |
---+---
2  |  |

```

Enter position as 'row col' (e.g., 0 2): 1 1

```

 0 1 2
0  |  |
---+---
1  | 0 |
---+---
2  |  |

```

Computer is making a move...

```

 0 1 2
0  |  |
---+---
1  | 0 |
---+---
2  |  | X

```

Enter position as 'row col' (e.g., 0 2): 0 2

```

 0 1 2
0  |  | 0
---+---
1  | 0 |
---+---
2  |  | X

```

Computer is making a move...

```

 0 1 2
0  |  | 0
---+---
1  | 0 |
---+---

```

```
2 X |   | X
```

Enter position as 'row col' (e.g., 0 2): 0 0

```
0 1 2
0 0 |   | 0
-----+-----
1   | 0 |
-----+-----
2 X |   | X
```

Computer is making a move...

```
0 1 2
0 0 |   | 0
```